FireSense

Configuration Manual

Complete step-by-step guide to installing and connecting your RAK 4631 IoT node!

Version	**1.0**	**Platform**	**FireSense IoT**
Language	**English**	**Hardware**	**RAK4631 + RAK7289V2**
Date	**2026**	**Protocol**	**LoRaWAN EU868 OTA**

INDEX

1. Hardware and software requirements
2. Install and configure ChirpStack (your own server) — Option A
3. Configure the Arduino IDE environment
4. Obtain OTAA keys and program the RAK4631 firmware
5. Configure the RAK7289V2 gateway
6. Register the node in ChirpStack
7. ChirpStack — Option B: FireSense Cloud Server
8. Configure Node-RED (import flow automatically)
9. Direct InfluxDB integration to ChirpStack
10. Configure Grafana — Visual Dashboard
11. Link the node to the FireSense platform
12. Verification and troubleshooting
13. JSON Node-RED to import

✓ RECOMMENDED CONFIGURATION ORDER: First install ChirpStack (section 2), then program the firmware (section 4) with the keys you obtained from ChirpStack, then configure the gateway (section 5) with your known server IP.

1 Hardware and software requirements

Required hardware:

Component	Model	Function
Microcontroller	RAK4631 (nRF52840 + SX1262)	LoRaWAN sensor node
Base plate	RAK19007 WisBlock Base	Connect all modules without soldering
Air temperature/humidity sensor	RAK1901 (SHTC3)	Air temperature and humidity (I2C)
Soil moisture sensor	RAK12035	Soil moisture 0-100% (I2C)
LoRaWAN Gateway	RAK7289V2	LoRa radio bridge — internet
Battery	LiPo 3.7V (2x18650 recommended)	Independent feeding for months

Required software:

Software	Version	Download
Arduino IDE	2.x o superior	arduino.cc/en/software
BSP RAKwireless nRF	v1.3.3+	Via Arduino IDE Board Manager
Driver USB CH341	Last	wch-ic.com/downloads/CH341SER_EXE.html

2 Installing ChirpStack with Docker — Option A: Own server

✓ We install ChirpStack FIRST because we need its IP address and OTAA keys before flashing the firmware and configuring the gateway. If you're using the FireSense Cloud service, skip to section 7.

2.1 Server requirements

- Ubuntu 22.04 LTS or Debian 12 (recommended)
- Minimum 2GB RAM, 20GB of disk space
- Ports accessible from the gateway: 8080 (UI ChirpStack), 1883 (MQTT)
- Static IP or known domain (you will need this IP in the following steps)

2.2 Install Docker

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# Verify installation:
docker --version
docker compose version
```

2.3 Create directory structure

```
mkdir -p ~/chirpstack/{chirpstack,mosquitto,postgres,redis}
mkdir -p ~/chirpstack/chirpstack/regions
cd ~/chirpstack
```

2.4 Crear docker-compose.yml

```
cat > docker-compose.yml << 'EOF'
services:
  chirpstack:
    image: chirpstack/chirpstack:4
    container_name: chirpstack
    restart: unless-stopped
    command: -c /etc/chirpstack
    volumes:
      - ./chirpstack:/etc/chirpstack
    depends_on: [postgres, mosquitto, redis]
    ports:
      - "8080:8080"
    networks: [chirpstack-net]

  postgres:
    image: postgres:14-alpine
    container_name: chirpstack-postgres
    restart: unless-stopped
    environment:
      - POSTGRES_PASSWORD=chirpstack
      - POSTGRES_USER=chirpstack
      - POSTGRES_DB=chirpstack
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks: [chirpstack-net]

  redis:
    image: redis:7-alpine
    container_name: chirpstack-redis
    restart: unless-stopped
    volumes: [redis-data:/data]
    networks: [chirpstack-net]

  mosquitto:
    image: eclipse-mosquitto:2
    container_name: chirpstack-mqtt
    restart: unless-stopped
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto:/mosquitto/config
    networks: [chirpstack-net]

networks:
  chirpstack-net:
    driver: bridge

volumes:
  postgres-data:
return-data:
EOF
```

2.5 Crear chirpstack.toml

```
cat > chirpstack/chirpstack.toml << 'EOF'
[logging]
  level = "info"

[postgresql]
  dsn = "postgres://chirpstack:chirpstack@postgres/chirpstack?sslmode=disable"

[redis]
  servers = ["redis://redis/"]

[network]
net_id = "000000"
  enabled_regions = ["eu868"]

[api]
  bind = "0.0.0.0:8080"
```

```
secret = "CHANGE_TO_A_LONG_AND_SECURE_PASSWORD"

[integration]
  enabled = ["mqtt"]
  [integration.mqtt]
    server = "tcp://mosquitto:1883"
    json = true
EOF
```

2.6 Create eu868.toml — CRITICAL for viewing the gateway

⚠ **Attention:**Without the [regions.gateway] section, ChirpStack will not subscribe to the gateway's MQTT topics and will never see it online. This file is essential.

```
cat > chirpstack/regions/eu868.toml << 'EOF'
[[regions]]
  id = "eu868"
  description = "EU868"

  [regions.gateway]
    force_gws_private = false
    [regions.gateway.backend]
      enabled = "mqtt"
      [regions.gateway.backend.mqtt]
        event_topic   = "eu868/gateway/+/event/+"
        command_topic = "eu868/gateway/{{ gateway_id }}/command/{{ command }}"
        server        = "tcp://mosquitto:1883"
        clean_session = false
        qos           = 0

  [regions.network]
    installation_margin = 10
    rx_window = 0
    rx1_delay = 1
    rx2_dr = 0
    rx2_frequency = 869525000
min_dr = 0
max_dr = 5
uplink_max_eirp = 16

  [[regions.network.extra_channels]]
    frequency = 867100000
min_dr = 0
max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867300000
min_dr = 0
max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867500000
min_dr = 0
max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867700000
min_dr = 0
max_dr = 5
  [[regions.network.extra_channels]]
    frequency = 867900000
min_dr = 0
max_dr = 5
EOF
```

2.7 Create Mosquitto configuration

```
cat > mosquitto/mosquitto.conf << 'EOF'
listener 1883
allow_anonymous true
persistence true
```

```
persistence_location /mosquitto/data/
log_dest stdout
EOF
```

2.8 Start ChirpStack

```
cd ~/chirpstack
docker compose up -d

# View logs (expect to see "starting api server"):
docker compose logs -f chirpstack

# Verify that everyone is healthy:
docker compose ps
```

ℹ ChirpStack available at http://YOUR_IP:8080 — Username: admin — Password: admin. Change it immediately.

How do I find out what your IP address is?

Run this command on your server to see the IP address:

```
# On Linux/Mac:
ip addr show | grep "inet " | grep -v 127.0.0.1
# Example result: inet 192.168.1.50/24 — your IP is 192.168.1.50

# En Windows (PowerShell):
ipconfig | findstr IPv4
```

Write down this IP address. You will use it in the gateway (step 5.3) and in Node-RED (step 8.4).

2.9 Crear Device Profile

ChirpStack → Tenants → Default → Device Profiles → Add device profile:

Parameter	Value
Name	firesense-node-otaa
Region	EU868
MAC version	LoRaWAN 1.0.3
Regional parameters revision	A
ADR algorithm	Default ADR
Expected uplink interval	3600 seconds
Device supports OTAA	Activated ✓

2.10 Add JavaScript codec (7 bytes)

Device Profile → Codec tab → JavaScript functions → paste this code:

```
function decodeUplink(input) {
  var b = input.bytes;
  if (b.length < 7) return { data: {} };
var t = (b[0] << 8) | b[1];
if (t > 32767) t -= 65536; // negative temperature
var h = (b[2] << 8) | b[3];
var bat = (b[5] << 8) | b[6];
  return { data: {
    temperatura:   parseFloat((t / 100).toFixed(2)),
```

```
    humedad_aire:  parseFloat((h / 100).toFixed(2)),
soil_moisture: b[4],
battery_mv: one
  }};
}
```

⚠ **Attention:**The payload is 7 bytes. Always use b.length < 7. If you set it to == 5, the codec will not decode anything.

2.11 Crear Application

ChirpStack → Applications → Add application → Name: firesense-nodes → Submit.

✓ Note down the IP address of your ChirpStack server (the same one you used to access the panel). You will need it in steps 3, 5, and 8.

3 Configure the Arduino IDE environment

3.1 Install Arduino IDE

Download Arduino IDE 2.x from arduino.cc/en/software and install it on your operating system.

3.2 Add RAKwireless URL

Arduino IDE → File → Preferences → campo "Additional boards manager URLs" → añade:

```
https://raw.githubusercontent.com/RAKWireless/RAKwireless-Arduino-BSP-Index
/main/package_rakwireless_index.json
```

3.3 Instalar BSP RAKwireless

Tools → Board → Boards Manager → search for "RAK" → install RAKwireless nRF Boards (v1.3.3+). This takes several minutes.

3.4 Install libraries

Tools → Manage Libraries → install these 4:

- LoRaWan-RAK4630 — management LoRaWAN OTA
- Adafruit SHTC3 Library (v1.0.2+) — air temperature/humidity sensor
- RAK12035_SoilMoisture — soil moisture sensor
- Adafruit Unified Sensor — dependency of SHTC3 (required)

3.5 Select motherboard and port

Tools → Board → RAKwireless nRF Boards → WisBlock RAK4631. Connect the RAK4631 via USB → Tools → Port → select the port (COMx on Windows, /dev/ttyUSB0 on Linux).

4 Obtain OTAA keys and program the RAK4631 firmware

✓ Now that ChirpStack is up and running, we get the OTAA keys directly from its interface and put them into the firmware.

4.1 Obtain the Device EUI of the node

The Device EUI is printed on the adhesive label of the RAK4631 module. Format on the label: AC:1F:09:FF:FE:24:54:24

⚠ **Attention:**The firmware requires the DevEUI in C++ hexadecimal format, NOT with a colon. Convert it like this: each comma-separated pair with a 0x prefix.

Label format	Format and firmware
AC:1F:09:FF:FE:24:54:24	{0xAC, 0x1F, 0x09, 0xFF, 0xFE, 0x24, 0x54, 0x24}

4.2 Obtain the Application Key from ChirpStack

The Application Key (AppKey) is a 16-byte secret key. There are two options:

OPTION A — Generate a new AppKey in ChirpStack (recommended):

- ChirpStack → Applications → firesense-nodes → Add device
- Fill in: Name (e.g., rak-node-01), Device EUI (without colon)
- Device Profile: firesense-node-otaa → Submit
- Go to the OTAA Keys tab → click the dice icon to GENERATE a random AppKey
- Copy the generated AppKey → you will use it in the firmware

OPTION B — Use AppKey already recorded in the firmware:

- If the node already has an AppKey in the firmware, note it down.
- In ChirpStack → OTAA Keys → enter that same AppKey manually

✓ The firmware AppKey and the ChirpStack AppKey MUST match exactly. If they don't match, the node will never be able to perform the OTAA Join.

4.3 Full firmware code

Create a new sketch in the Arduino IDE. Replace the keys with your own:

```
#include <Arduino.h>
#include <LoRaWan-RAK4630.h>
#include <SPI.h>
#include <Wire.h>
#include "Adafruit_SHTC3.h"
#include <RAK12035_SoilMoisture.h>

# REPLACE: DevEUI of the label (format 0xXX)
uint8_t nodeDevEui[] = {0xAC, 0x1F, 0x09, 0xFF, 0xFE, 0x24, 0x54, 0x24};
uint8_t nodeAppEui[] = {0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00};
```

```
# REPLACE: AppKey obtained from ChirpStack (step 4.2)
uint8_t nodeAppKey[] = {0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX,
                        0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX, 0xXX};

Adafruit_SHTC3 shtc3;
RAK12035 soil_sensor;
bool joined = false;
#define vbat_pin        A0
#define VBAT_MV_PER_LSB 1.279f

void lorawan_rx_handler(lmh_app_data_t *d) {}
void lorawan_has_joined_handler(void) { joined = true; }
void lorawan_join_failed_handler(void) {}
void lorawan_confirm_class_handler(DeviceClass_t C) {}
static lmh_callback_t lora_callbacks = {
  BoardGetBatteryLevel, BoardGetUniqueId, BoardGetRandomSeed,
  lorawan_rx_handler, lorawan_has_joined_handler,
  lorawan_confirm_class_handler, lorawan_join_failed_handler };
static lmh_param_t lora_param_init = {
  LORAWAN_ADR_ON, DR_3, LORAWAN_PUBLIC_NETWORK, 8,
  LORAWAN_DEFAULT_DATARATE, CLASS_A };

float readVBAT() { return analogRead(vbat_pin) * VBAT_MV_PER_LSB; }
uint8_t mvToPercent(float mv) {
  if (mv>=4200) return 100; if (mv>=4000) return 85;
  if (mv>=3800) return 60;  if (mv>=3600) return 30;
  if (mv>=3400) return 10;  return 0; }

void setup() {
  pinMode(WB_IO2, OUTPUT);
digitalWrite(WB_IO2, HIGH); # CRITICAL: turns on sensor power
  delay(500);
  analogReference(AR_INTERNAL_3_0);
  analogReadResolution(12);
  delay(5); readVBAT(); readVBAT();
  Wire.begin(); shtc3.begin(); soil_sensor.begin();
  lora_rak4630_init();
  lmh_setDevEui(nodeDevEui);
  lmh_setAppEui(nodeAppEui);
  lmh_setAppKey(nodeAppKey);
  lmh_init(&lora_callbacks, lora_param_init, true, CLASS_A, LORAMAC_REGION_EU868);
  lmh_join();
}

void loop() {
  if (!joined) { delay(1000); return; }
  sensors_event_t hum, temp;
  shtc3.getEvent(&hum, &temp);
  uint8_t moisture = 0;
  soil_sensor.get_sensor_moisture(&moisture);
  float bat = (readVBAT()+readVBAT()+readVBAT())/3.0f;
  uint16_t batInt = (uint16_t)bat;
  int16_t  t = (int16_t)(temp.temperature * 100);
  uint16_t h = (uint16_t)(hum.relative_humidity * 100);
# Payload 7 bytes: temp(2)+humid_air(2)+humid_soil(1)+bat_mv(2)
  uint8_t payload[7];
  payload[0]=(uint8_t)(t>>8);      payload[1]=(uint8_t)t;
  payload[2]=(uint8_t)(h>>8);      payload[3]=(uint8_t)h;
  payload[4]=moisture;
  payload[5]=(uint8_t)(batInt>>8); payload[6]=(uint8_t)batInt;
  lmh_app_data_t d; d.port=1; d.buffsize=7; d.buffer=payload;
  lmh_send(&d, LMH_UNCONFIRMED_MSG);
delay(3600000); # Sleep for 1 hour
}
```

4.4 Upload the firmware

Click Upload (→). This will take 1-2 minutes. Open Serial Monitor at 115200 baud. You will see Join Request and Join Accept if ChirpStack is running and the gateway is online.

⚠ **Attention:**If WB_IO2 returns an error, the BSP RAKwireless is not installed. Repeat step 3.3.

✓ If you see JoinRequest in the Serial Monitor but do NOT see JoinAccept, this is completely normal at this point. The gateway is not yet configured. Continue to section 5, and the JoinAccept will arrive when the gateway is online.

5 Configure the RAK7289V2 gateway

✓ You now have the IP address of your ChirpStack server (from step 2). You will use it here as your MQTT Broker Address.

5.1 Connect the gateway to the internet

Connect to your router via Ethernet or activate the integrated 4G. Wait 2 minutes (solid green LED).

5.2 Access the web interface

```
URL:       http://192.168.1.1
User: root
Password: root
```

ℹ If you can't access it, connect a direct Ethernet cable between your PC and gateway. Your PC will automatically obtain the IP address 192.168.1.x.

5.3 Configure MQTT Bridge

LoRa → LoRa Network → Network Settings:

Parameter	Value
Protocol	LoRa Gateway MQTT Bridge
MQTT Protocol Version	MQTT for ChirpStack 4.x (Protobuf)
MQTT Broker Address	YOUR CHIRPSTACK SERVER IP (from step 2)
MQTT Broker Port	1883
SSL/TLS	None (disabled)
User Authentication	Deactivated

5.4 WireGuard VPN — Private network only

⚠ **Attention:**Only if the gateway is on a private network without direct access to the server. If you have direct internet access, skip to step 5.5.

Extensions → RAK WireGuard → Install. Configure with your administrator credentials:

- Address: IP address of the gateway in the tunnel (e.g., 10.0.0.28/24)
- Endpoint: VPN server IP:port (e.g., server.company.com:443)
- AllowedIPs: Red del servidor (ej: 10.2.0.0/16)

- PersistentKeepalive: 25000 ms

5.5 Check gateway online

ChirpStack → Gateways: green dot and "Last seen" a few seconds ago.

If Offline appears: check the MQTT Broker IP, port, and network connectivity.

6 Register the node in ChirpStack

✓ Check in ChirpStack → Applications → firesense-nodes → your device → LoRaWAN Frames that you see JoinRequest + JoinAccept. If you see them, the node is registered and working correctly. If the device doesn't exist yet, follow the steps below.

6.1 Add the device to ChirpStack

ChirpStack → Applications → firesense-nodes → Add device:

- Name: rak-node-01 (or the name you prefer)
- Device EUI: the one on the RAK4631 label, WITHOUT a colon (e.g., ac1f09fffe245424)
- Device Profile: firesense-node-otaa
- → Submit

6.2 Configure OTAA Keys

OTAA Keys tab → Enter the Application Key. It must be EXACTLY the same one you entered in the firmware (step 4.2). → Submit.

6.3 Verify Join OTAA

Power on the RAK4631 node. In ChirpStack → Device → LoRaWAN Frames, you should see:

```
JoinRequest ← the node requests to join
JoinAccept → ChirpStack accepts the connection
(appears in a few seconds if the gateway is online)
```

If you see JoinRequest but not JoinAccept: the AppKey does not match between firmware and ChirpStack.

7 ChirpStack — Option B: FireSense Cloud Server

If you subscribe to the FireSense Cloud service, the server is already configured. You will receive your credentials by email.

7.1 Access the panel

```
URL:        https://chirpstack.firesense.io
Username: your_email@company.com
Password: (the one received by email)
```

7.2 Register node and gateway

The Device Profile has already been created. Go to Applications → firesense-nodes → Add device. Follow steps 6.1 to 6.3.

For the gateway, use the following MQTT Broker Address: mqtt.firesense.io, port 1883.

8 Configure Node-RED

Node-RED receives data from ChirpStack and stores it in InfluxDB. You need your InfluxDB server's IP address and access token.

8.1 Obtain the InfluxDB token

Access InfluxDB at http://YOUR_IP:8086 → Data → API Tokens → Generate API Token → All Access Token. Copy the generated token.

8.2 Access Node-RED

```
http://TU_IP:1880/nodered/
```

8.3 Import the flow

Menu ≡ → Import → paste the JSON from section 13 → Import → Deploy (red button).

8.4 Configure credentials

Double-click on the "InfluxDB" node:

- URL: http://influxdb:8086 (o http://TU_IP_INFLUXDB:8086)
- Token: the token copied in step 8.1
- Organization: Your organization
- Bucket: sensor_data

Double-click on the "MQTT ChirpStack" node → Edit Broker:

> ⚠ **Attention:**Replace 'localhost' with the actual IP address of your ChirpStack server. If Node-RED and ChirpStack are on the same server, you can leave it as 'localhost' or 'mosquitto'.

→ Deploy to save.

9 Direct InfluxDB integration to ChirpStack

Alternative to Node-RED: ChirpStack can send data directly to InfluxDB without intermediaries.

ℹ You can use Node-RED (section 8) or this direct integration. You don't need to use both. Direct integration is simpler; Node-RED allows for more advanced processing.

9.1 Activate integration

ChirpStack → Applications → firesense-nodes → ... tab → InfluxDB → + button:

Parameter	Value
Endpoint	http://TU_IP:8086
Version	InfluxDB v2
Token	InfluxDB administrator token
Organization	Your organization
Bucket	sensor_data

→ Submit. ChirpStack will automatically write all codec fields to InfluxDB.

10 Configure Grafana — Visual Dashboard

10.1 Access Grafana

```
URL:        http://TU_IP:3000
User: admin
Password: admin (change on first login)
```

10.2 Add InfluxDB as a Data Source

Configuration → Data Sources → Add data source → InfluxDB:

Parameter	Value
Name	InfluxDB FireSense
Query Language	Flux
URL	http://influxdb:8086 (o http://TU_IP:8086)
Organization	Your organization
Token	InfluxDB administrator token
Default Bucket	sensor_data

→ Save & Test → debe aparecer "datasource is working".

10.3 Enable embedding for the FireSense website

To display Grafana panels within the FireSense map, add the following to your Grafana docker-compose:

```
environment:
  - GF_AUTH_ANONYMOUS_ENABLED=true
  - GF_AUTH_ANONYMOUS_ORG_ROLE=Viewer
  - GF_SECURITY_ALLOW_EMBEDDING=true
```

Restart: docker compose restart grafana

10.4 Import FireSense dashboard

Grafana → Dashboards → Import → paste this JSON from the dashboard:

```
{"title":"FireSense — Nodes
IoT","uid":"firesense-main","tags":["firesense","iot"],"timezone":"Europe/Madrid",
"panels":[
  {"id":1,"type":"timeseries","title":"Temperatura
(°C)","gridPos":{"x":0,"y":0,"w":12,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"temperatura\")"}],
   "fieldConfig":{"defaults":{"unit":"celsius","color":{"mode":"palette-classic"}}}},
{"id":2,"type":"timeseries","title":"Air Humidity
(%)","gridPos":{"x":12,"y":0,"w":12,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"humedad_aire\")"}],
   "fieldConfig":{"defaults":{"unit":"percent","min":0,"max":100}}},
{"id":3,"type":"timeseries","title":"Soil Humidity
(%)","gridPos":{"x":0,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -24h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"humedad_suelo\")"}],
   "fieldConfig":{"defaults":{"unit":"percent","min":0,"max":100}}},
  {"id":4,"type":"gauge","title":"Bateria (%)","gridPos":{"x":8,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -1h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"bateria_mv\")\n  |> last()"}],

"fieldConfig":{"defaults":{"unit":"mV","thresholds":{"steps":[{"color":"red","value":0}
,{"color":"yellow","value":3600},{"color":"green","value":3800}]}}}},
  {"id":5,"type":"stat","title":"RSSI","gridPos":{"x":16,"y":8,"w":8,"h":8},
   "targets":[{"datasource":{"type":"influxdb"},"query":"from(bucket:\"sensor_data\")\n
|> range(start: -1h)\n  |> filter(fn: (r) => r._measurement == \"espurna_sensors\")\n
|> filter(fn: (r) => r._field == \"rssi\")\n  |> last()"}],

"fieldConfig":{"defaults":{"unit":"dBm","thresholds":{"steps":[{"color":"red","value":-
120},{"color":"yellow","value":-100},{"color":"green","value":-80}]}}}}
],"time":{"from":"now-24h","to":"now"},"refresh":"1m","schemaVersion":38}
```

After importing → in each panel → Edit → Query → Data source → select "InfluxDB FireSense" → Save dashboard.

11 Link the node to the FireSense platform

✓ This is the final step. The node is now sending data to InfluxDB. Now you link it to your FireSense account so it appears on the map.

11.1 Log in to FireSense

```
URL: https://YOUR_DOMAIN/FireSense/login.html
```

11.2 Open the wizard and link the Device EUI

On the dashboard → "Telemetry in Viu" bar → + ADD NODE button (top right).

The wizard has 4 steps. In step 4, enter the Device EUI in 16-character hexadecimal format WITHOUT a colon:

Correct format	INCORRECT format
ac1f09fffe245424	AC:1F:09:FF:FE:24:54:24

→ Link node. The node will appear on the map automatically.

ℹ You can add as many nodes as you like by repeating this process. Each node appears on the map with an automatic ID (RAK-01, RAK-02...) along with its temperature, humidity, and battery data.

12 Verification and troubleshooting

Checklist:

OK	Verification
[ ]	ChirpStack accesible en http://TU_IP:8080
[ ]	Gateway appears Online in ChirpStack (green dot under Gateways)
[ ]	Node performs OTAA Join (JoinRequest + JoinAccept on Device → LoRaWAN Frames)
[ ]	Data decoded in ChirpStack → Device → Events
[ ]	Node-RED receives data (debug node displays object with fields)
[ ]	InfluxDB contains data in the sensor_data bucket
[ ]	Grafana displays graphs (Data Source works)
[ ]	Linked node visible on the FireSense map

Common problems:

Problem	Solution
Gateway Offline en ChirpStack	Verify the MQTT Broker IP address (it should be the ChirpStack server IP address). Verify port 1883.
Node does not perform JoinAccept	AppKey in firmware and ChirpStack do not match. Verify byte by byte.
JoinRequest but not JoinAccept	The gateway is not online or the AppKey does not match.
Sensors return 0 or 255	digitalWrite(WB_IO2, HIGH) is missing from setup(). It is the most important line in the firmware.
Codec no decoding datos	Verify that it uses b.length < 7 (7-byte payload). Do not use == 5.
There is no data in InfluxDB	Verify InfluxDB and Node-RED tokens. The tag in the JSON should be dev_eui (not node_id).
Node does not appear on the map	Verify that the DevEUI registered in FireSense matches the one used by the node (without uppercase letters or colons).

13 JSON Node-RED — Complete flow for importing

✓ IMPORTANT: After importing the JSON, there are TWO settings you need to change: (1) In the 'MQTT ChirpStack' node — double-click — Edit Broker — Server: change 'localhost' to the IP address of your ChirpStack server. (2) In the 'InfluxDB' node — double-click — change the URL, token, and organization. The rest of the JSON does not need to be modified.

```
[
  {
    "id": "firesense-main-tab",
    "type": "tab",
    "label": "FireSense LoRaWAN",
    "disabled": false
  },
  {
    "id": "chirpstack-mqtt-in",
    "type": "mqtt in",
    "z": "firesense-main-tab",
    "name": "ChirpStack MQTT",
    "topic": "application/+/device/+/event/up",
    "qos": "0",
    "datatype": "json",
    "broker": "chirpstack-broker-cfg",
"x": 130, "y": 100,
    "wires": [["fn-decode-payload"]]
  },
  {
    "id": "fn-decode-payload",
    "type": "function",
    "z": "firesense-main-tab",
"name": "Decode 7 bytes",
    "func": "var b=msg.payload.data;if(!b)return null;var
bytes=Buffer.from(b,'base64');if(bytes.length<7)return null;var
t=(bytes[0]<<8)|bytes[1];if(t>32767)t-=65536;var h=(bytes[2]<<8)|bytes[3];var
bat=(bytes[5]<<8)|bytes[6];msg.payload={measurement:'espurna_sensors',tags:{dev_eui:msg.payload.deviceInfo.
devEui,device_name:msg.payload.deviceInfo.deviceName},fields:{temperatura:parseFloat((t/100).toFixed(2)),hu
medad_aire:parseFloat((h/100).toFixed(2)),humedad_suelo:bytes[4],bateria_mv:bat,rssi:(msg.payload.rxInfo&&m
sg.payload.rxInfo[0])?msg.payload.rxInfo[0].rssi:0,snr:(msg.payload.rxInfo&&msg.payload.rxInfo[0])?msg.payl
oad.rxInfo[0].snr:0,fcnt:msg.payload.fCnt||0},timestamp:new Date().getTime()*1000000};return msg;",
"x": 330, "y": 100,
    "wires": [["influxdb-write-out","debug-datos"]]
  },
  {
    "id": "influxdb-write-out",
    "type": "influxdb out",
    "z": "firesense-main-tab",
    "influxdb": "influxdb-v2-cfg",
"name": "Save to InfluxDB",
"measurement": "spark_sensors",
    "precision": "ns",
"x": 560, "y": 80,
    "wires": []
  },
  {
"id": "debug-data",
    "type": "debug",
    "z": "firesense-main-tab",
    "name": "Debug",
    "active": true,
    "complete": "payload",
"x": 560, "y": 120,
    "wires": []
  },
  {
    "id": "chirpstack-broker-cfg",
    "type": "mqtt-broker",
    "name": "ChirpStack MQTT",
    "broker": "localhost",
    "port": "1883",
    "clientid": "nodered-firesense",
    "autoConnect": true,
    "usetls": false,
    "protocolVersion": "4",
    "keepalive": "60",
    "cleansession": true
  },
  {
    "id": "influxdb-v2-cfg",
    "type": "influxdb",
    "hostname": "localhost",
    "port": "8086",
    "protocol": "http",
    "name": "InfluxDB v2 FireSense",
    "influxdbVersion": "2.0",
    "url": "http://localhost:8086",
    "rejectUnauthorized": false
  }
]
```